

Sistemas Distribuídos — Aula 18

Algoritmos Distribuídos I: Consenso Distribuído · 13/10 · Prof. Leandro Borges

O problema do consenso distribuído

Em um sistema distribuído, múltiplos processos frequentemente precisam concordar sobre um único valor — qual é o coordenador atual, qual foi a última operação confirmada, qual réplica contém os dados corretos, ou até mesmo se uma mensagem foi entregue. Esse acordo é o que chamamos de consenso: um protocolo que garante que um conjunto de processos, mesmo operando de forma independente e trocando mensagens por uma rede que pode atrasar, perder ou entregar fora de ordem, chegue a uma única decisão final compartilhada.

O consenso é a base de praticamente todo mecanismo de tolerância a falhas em sistemas reais: eleição de líder, replicação de log em bancos de dados distribuídos, coordenação de transações e serviços de configuração distribuída (como o ZooKeeper ou o etcd) dependem, no fundo, de algum algoritmo de consenso rodando por trás. Um algoritmo de consenso correto precisa garantir três propriedades: acordo (todos os processos corretos decidem o mesmo valor), validade (o valor decidido foi realmente proposto por algum processo) e terminação (todo processo correto eventualmente decide um valor, em tempo finito).

Por que consenso é difícil: a impossibilidade de FLP

Em 1985, Fischer, Lynch e Paterson publicaram um resultado teórico que se tornou fundamental para toda a área de sistemas distribuídos, conhecido pela sigla FLP. O resultado prova que, em um sistema assíncrono — onde não há limite conhecido para o tempo de entrega de mensagens nem para a velocidade de processamento — é impossível garantir consenso determinístico com terminação garantida, mesmo que apenas um único processo possa falhar (por parada, sem nenhum comportamento malicioso).

Isso não significa que consenso seja impraticável: significa que qualquer algoritmo real precisa abrir mão de alguma garantia teórica absoluta para funcionar na prática. A saída usada por algoritmos como Paxos e Raft é sacrificar a terminação garantida em casos extremos (o algoritmo pode, em teoria, nunca terminar se a rede for suficientemente hostil) em troca de manter sempre a correção: o sistema nunca decide um valor errado, mesmo que em situações raras ele demore para decidir algum valor.

Paxos: o algoritmo clássico

Paxos, proposto por Leslie Lamport, foi o primeiro algoritmo de consenso amplamente aceito como corretamente demonstrado e é a base teórica da maioria dos sistemas de consenso usados hoje. Ele funciona em duas fases principais. Na fase Prepare/Promise, um processo proponente escolhe um número de proposta e o envia a uma maioria dos aceitadores, pedindo que eles prometam não aceitar propostas mais antigas; cada aceitador responde com a proposta de maior número que já aceitou, se houver alguma.

Na fase Accept/Accepted, o proponente — já sabendo se alguma proposta anterior foi parcialmente aceita — envia um valor para essa mesma maioria de aceitadores, que o aceita se ainda não prometeu a nenhuma proposta mais recente. Quando uma maioria aceita o mesmo valor, o consenso está decidido. A exigência de maioria (quorum) é o que garante que duas decisões conflitantes nunca aconteçam: quaisquer duas maiorias sempre têm pelo menos um aceitador em comum.

Paxos vs. Raft: duas abordagens ao mesmo problema

Apesar de correto, Paxos é notoriamente difícil de entender e de implementar corretamente — sua descrição original é abstrata e deixa várias decisões de engenharia em aberto, o que levou a implementações incompatíveis entre si ao longo dos anos. Foi motivado por essa dificuldade que Diego Ongaro e John Ousterhout propuseram, em 2014, o Raft: um algoritmo com as mesmas garantias de correção de Paxos, mas desenhado explicitamente para ser mais fácil de compreender e implementar.

A diferença central está na estrutura: enquanto Paxos trata cada decisão como uma negociação simétrica entre processos, Raft elege explicitamente um líder único, que centraliza a replicação do log de operações enquanto permanecer ativo — só quando o líder falha (detectado por timeout) é que uma nova eleição ocorre entre os seguidores. Essa divisão clara entre eleição de líder, replicação de log e segurança torna o comportamento do Raft muito mais previsível e didático, sem abrir mão da tolerância a falhas.

Estudo de caso: consenso em sistemas reais

O etcd é um armazenamento chave-valor distribuído, usado pelo Kubernetes para guardar todo o estado do cluster (configuração, estado dos pods, segredos). Ele usa Raft internamente para replicar cada escrita entre um número ímpar de réplicas e eleger um líder — se o líder cair, uma nova eleição acontece automaticamente em segundos, sem intervenção manual e sem perda de dados já confirmados por uma maioria.

O Apache ZooKeeper, por sua vez, é usado como serviço de coordenação por Kafka, Hadoop e diversos outros sistemas distribuídos. Ele utiliza o protocolo ZAB (ZooKeeper Atomic Broadcast), inspirado nas ideias de Paxos, para manter todas as réplicas de configuração consistentes. Em ambos os casos, o padrão se repete: em vez de cada

sistema reinventar seu próprio algoritmo de consenso, ele delega essa responsabilidade a uma camada dedicada, testada e comprovadamente correta.

Referências

LAMPORT, L. The Part-Time Parliament. ACM Transactions on Computer Systems, v. 16, n. 2, 1998.

ONGARO, D.; OUSTERHOUT, J. In Search of an Understandable Consensus Algorithm (Raft). USENIX Annual Technical Conference, 2014.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.